

1 Prompt

```
def winner(self, token):
    directions = [(0, 1), (1, 0)]
    for r in range(ROWS):
        for c in range(COLS):
            if self.grid[r][c] != token:
                continue
            for dr, dc in directions:
                if all(0 <= r + i * dr < ROWS and 0 <= c + i * dc < COLS and self.grid[r + i * dr][c + i * dc] == token for i in range(4)):
                    return True
    return False
```

In board.py, update the winner method to detect 4-in-a-row in all four directions: horizontal (0, 1), vertical (1, 0), diagonal down-right (1, 1), and diagonal down-left (1, -1). Ensure shorter sequences do not count as wins.

Simulating game moves without altering the actual board.
Simulating game moves without altering the actual board.

There are still no Python files in this workspace, so here is the upgraded method to drop into ai.py rather than an edit:

```
python
import copy
import random
COLS = 7
class AI:
    # keep your existing class name
    def choose_column(self, board, me="O", opponent="X"):
        legal = [c for c in range(COLS) if board.grid[0][c] == "."]
        if not legal:
            return None
        # 1. Take an immediate win.
        for col in legal:
            if self._wins_if_played(board, col, me):
                return col
        # 2. Block the opponent's immediate win.
        for col in legal:
            if self._wins_if_played(board, col, opponent):
                return col
        # 3. Choose safely: avoid moves that hand the opponent a win
        # on top of our piece, and prefer central columns.
        safe = [c for c in legal if not self._gives_win_above(board, c, me, opponent)]
        candidates = safe or legal
        # if every move is unsafe, still play center
        COLS // 2
        best = min(abs(c - center) for c in candidates)
        return random.choice([c for c in candidates if abs(c - center) == best])
    @staticmethod
    def _wins_if_played(board, col, token):
        """True if dropping token in col wins. Works on a copy; the real board is untouched."""
        trial = copy.deepcopy(board)
        if trial.drop(col, token) is None:
            return False
        return trial.winner(token)
    @staticmethod
    def _gives_win_above(board, col, me, opponent):
        """True if playing col lets the opponent win by playing the same column next."""
        trial = copy.deepcopy(board)
        if trial.drop(col, me) is None:
            return False
        if trial.grid[0][col] != ".":
            # column now full, no reply possible there
            return False
        return trial.drop(col, opponent) is not None and trial.winner(opponent)
```

How it works:

Win, then block: it tries each legal column for the AI first, then for the opponent. It plays the first column that wins, or blocks if the opponent could win there. Winning is checked before blocking, so the AI never blocks when it could simply win.

Safe fallback: it skips any column where the AI's piece would let the opponent win by stacking on top of it. If every move is unsafe, it plays a legal one anyway rather than returning nothing. Among the safe moves it prefers the center columns, which are stronger in Connect Four, and picks randomly among ties.

Full columns: only columns whose top cell is "." are considered, and drop returning None is treated as "not a win". If the board is full the method returns None, which the game loop already handles.

No board mutation: each trial runs on copy.deepcopy(board), because I don't know whether your Board has an undo method. If it has a cheap undo or a copy method, swap that in for speed. With 7 columns the deepcopy cost doesn't matter.

This relies on `board.drop(col, token)` returning `None` for a full column and `board.winner(token)` being the four-direction version from earlier. If you attach `ai.py` and the board file, I can apply the change and test it against a few board positions.

```
from board import Board, COLS
from ai import AI
class Game:
    def __init__(self):
        self.board = Board()
        self.ai = AI()
        self.turn = "X"
    def run(self):
        print("Connect Four ? you are X.")
        while True:
            self.board.print()
            if self.turn == "X":
                try:
                    raw = input("Column (1-7), or q: ")
                except (EOFError, KeyboardInterrupt):
                    print("\nGoodbye.")
                    return
                if raw == "q":
                    print("Game quit.")
                    return
                try:
                    col = int(raw) - 1
                except ValueError:
                    print("Please enter a number from 1 to 7, or q to quit.")
                    continue
                if not 0 <= col < COLS:
                    print("Column must be between 1 and 7.")
                    continue
                else:
                    col = self.ai.choose_column(self.board)
                if col is None or not 0 <= col < COLS:
                    print("AI returned an invalid column; ending game.")
                    return
                if self.board.drop(col, self.turn) is None:
                    if self.turn == "X":
                        print(f"Column {col + 1} is full. Choose another.")
                        continue
                    print("AI chose a full column; ending game.")
                    return
                if self.board.winner(self.turn):
                    self.board.print()
                    print(self.turn, "wins!")
                    return
                if self.board.full():
                    self.board.print()
                    print("Draw.")
                    return
                self.turn = "O" if self.turn == "X" else "X"
```

In `game.py`, add concise move-level feedback that prints which column the player or AI successfully placed their disc into, appearing exactly once per actual move

Adding move-announcement messages for player and AI turns.

Adding move-announcement messages for player and AI turns.

Still no files in this workspace, so here is the change for your `game.py`. Add one print right after the successful drop, and nothing else needs to move:

```
python
if self.board.drop(col, self.turn) is None:
    if self.turn == "X":
        print(f"Column {col + 1} is full. Choose another.")
    continue
    print("AI chose a full column; ending game.")
    return
    who = "You" if self.turn == "X" else "AI"
    print(f"{who} placed a disc in column {col + 1}.")
    if self.board.winner(self.turn):
        self.board.print()
        print(self.turn, "wins!")
    return
```

The message prints exactly once per move, for these reasons:

Only after success: it sits after the failed-drop branch, which continues or returns.

Full-column attempts, invalid input and AI errors never reach it.

Before win and draw checks: the winning or drawing move is announced too, then the final board and result follow.

Not in the loop header: a re-prompt after bad input doesn't repeat any move message.

Column numbering: it prints `col + 1` so it matches the 1-7 numbers the player types.

Example output:

You placed a disc in column 4. AI placed a disc in column 4.

If you'd rather show X and O than "You" and "AI", replace the `who` line with `who = self.turn`.

[Claude is AI and can make

mistakes.](<https://support.anthropic.com/en/articles/8525154-claude-is-providing-incorrect-or-misleading-responses-what-s-going-on>)